

UNE TECHNIQUE DE TESTS BOÎTE NOIRE : LE TEST DE TRANSITION D'ÉTATS

Table des matières

1. Syllabus ISTQB Foundation v.4 4.2.4	1
2. Un exemple concret.....	3
3. Transitions invalides	7
4. Couverture 1-switch	8

1. Syllabus ISTQB Foundation v.4 4.2.4

Un diagramme de transition d'états *modélise le comportement d'un système* en montrant ses *états possibles et les transitions d'états valides*.

Une *transition* est initiée par un *événement*, qui peut être qualifié par une *condition de garde*.¹

Les transitions sont supposées être instantanées et peuvent parfois entraîner une action du logiciel.

La syntaxe courante utilisée pour désigner les transitions est la suivante : "événement [condition de garde] / action".

Les conditions de garde et les actions peuvent être omises si elles n'existent pas ou si elles ne sont pas pertinentes pour le testeur.

Une *table d'états* est un modèle équivalent à un *diagramme de transition d'états*.

Ses *lignes représentent les états* et ses *colonnes les événements* (ainsi que les conditions de garde si elles existent).

Les entrées de la table (cellules) représentent les transitions et contiennent l'état cible, ainsi que les conditions de garde et les actions résultantes, si elles sont définies.

Contrairement au diagramme de transition d'états, la table d'états montre explicitement les transitions non valides, qui sont représentées par des cellules vides.

Un cas de test basé sur un diagramme de transition d'état ou une table d'états est généralement représenté comme une séquence d'événements, qui résulte en une

¹ Par exemple, dans un système de connexion, un événement de "tentative de connexion" peut être accompagné de la condition de garde "mot de passe valide" : cette transition vers un état "connecté" ne se produira que si le mot de passe est correct, même si l'événement de connexion est déclenché.

On le notera : tentative de connexion [mot de passe valide] / connecté

séquence de changements d'état (et d'actions, si nécessaire).

Un cas de test peut, et c'est généralement le cas, couvrir plusieurs transitions d'états.

Il existe de nombreux critères de couverture pour les tests de transition d'état. Ce syllabus discute de trois d'entre eux.

Dans la couverture de tous les états, les éléments de couverture sont les états. Pour obtenir une couverture de 100 %, les cas de test doivent garantir que tous les états sont visités. La couverture est mesurée comme le nombre d'états visités divisé par le nombre total d'états, et est exprimée en pourcentage.

Dans la couverture des transitions valides (également appelée couverture 0-switch)², les éléments de couverture sont les transitions valides uniques. Pour obtenir une couverture de 100% des transitions valides, les cas de test doivent exercer toutes les transitions valides. La couverture est mesurée comme le nombre de transitions valides exercées divisé par le nombre total de transitions valides, et est exprimée en pourcentage.

Dans la couverture de toutes les transitions, les éléments de couverture sont toutes les transitions figurant dans une table d'état. Pour obtenir une couverture de 100 % de toutes les transitions, les cas de test doivent exercer toutes les transitions valides et tenter d'exécuter des transitions non valides.

Le fait de ne tester qu'une seule transition non valide dans un seul cas de test permet d'éviter le masquage des défauts, c'est-à-dire une situation dans laquelle un défaut empêche la détection d'un autre.

La couverture est mesurée comme le nombre de transitions valides et non valides exercées ou tentées d'être exercées par les cas de test exécutés, divisé par le nombre total de transitions valides et non valides, et est exprimée en pourcentage.

La couverture de tous les états est plus faible que la couverture des transitions valides, car elle peut généralement être obtenue sans exercer toutes les transitions.

La couverture des transitions valides est le critère de couverture le plus largement utilisé.

L'obtention d'une **couverture complète des transitions valides** garantit une couverture complète de tous les états.

L'obtention d'une **couverture complète de toutes les transitions** garantit à la fois une couverture complète de tous les états et une couverture complète des transitions valides et devrait constituer une exigence minimale pour les logiciels critiques en termes de mission et de sécurité.

² Ou 0-aiguillage en français

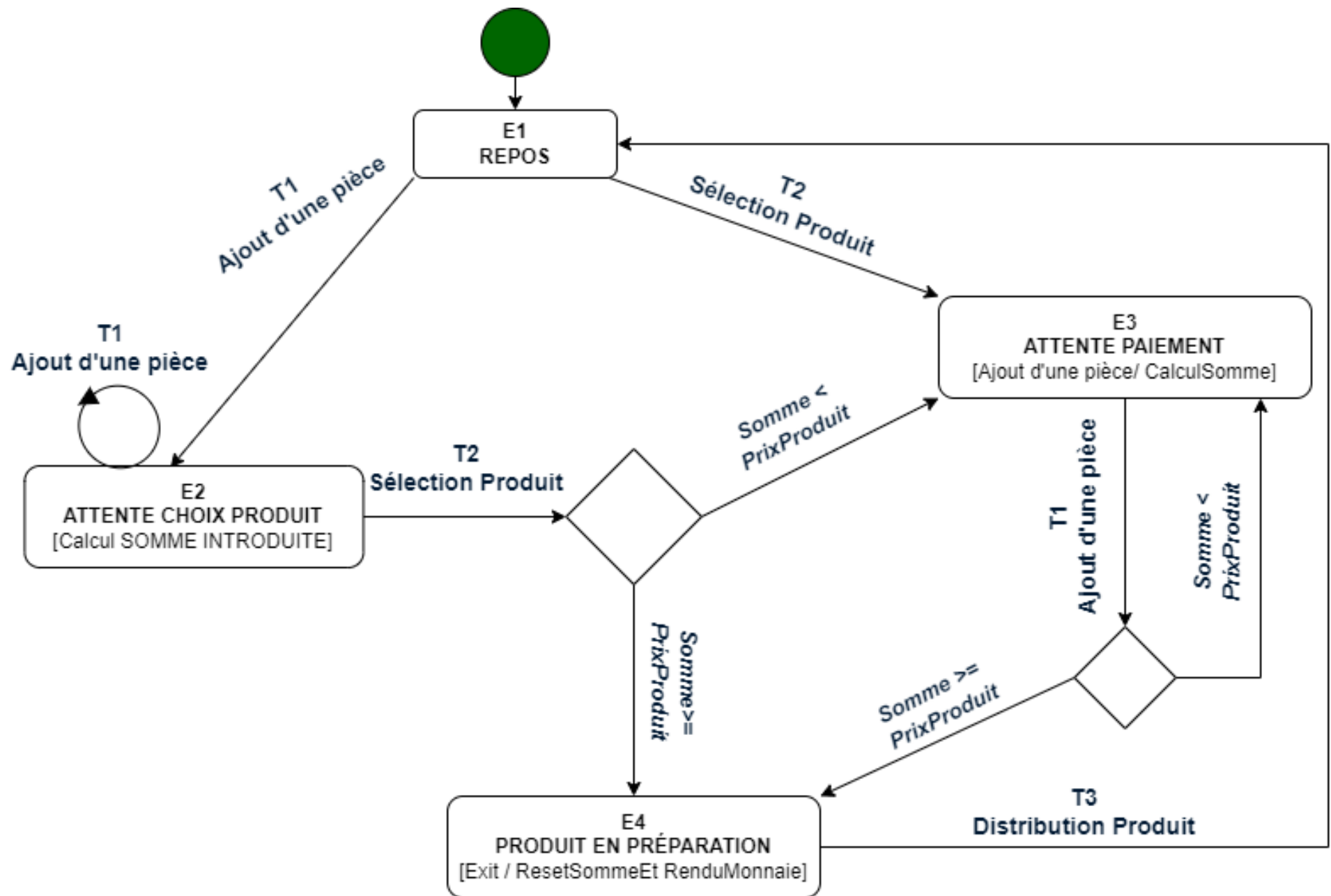
2. Un exemple concret

Un distributeur de boissons sera un très bon exemple pour une illustrer cette technique.

Prenons le distributeur suivant (volontairement simple et ... mal conçu):

- Le distributeur permet au client de choisir parmi plusieurs boissons en tapant un chiffre.
- Le distributeur accepte uniquement des pièces de monnaies.
- le distributeur prend en compte les pièces de monnaies uniquement avant et après la sélection d'une boisson par l'utilisateur.
- lorsque le montant de la boisson est atteint ou dépassé, le distributeur procède à la préparation de la boisson puis rend la monnaie s'il y a lieu.

Voici une proposition de diagramme d'état-transition pour cet énoncé :



Nous allons chercher à obtenir une couverture 0-switch (aiguillage-0) :
Toute séquence valide d'une seule transition sera testée au moins une fois.

Pour cela, deux possibilités:

- soit partir directement du graphe en créant les cas de test qui permettent de couvrir chaque transition partant de chaque état, en prenant soin de considérer chaque condition d'une transition conditionnelle de manière unique.
- soit réaliser la table de transition d'état et créer des cas de test pour couvrir chaque case indiquant une transition valide.

(NB : Pour plus de lisibilité, nous noterons les transitions invalides par un **X**)

	T1 AJOUT D'UNE PIÈCE		T2 AJOUT D'UNE PIÈCE		T3 DISTRIBUTION PRODUIT
	[Somme >= PrixProduit]	[Somme < PrixProduit]	[Somme >= PrixProduit]	[Somme < PrixProduit]	-
E1 REPOS	E2	E2	E3	E3	X
E2 ATTENTE PRODUIT	E2	E2	E4	E3	X
E3 ATTENTE PAIEMENT	E4	E3	X	X	X
E4 PRODUIT EN PRÉPARATION	X	X	X	X	E1

Nous avons 8 transitions valides que nous pouvons couvrir avec 3 cas de test, amenant de E1 à E1 en passant par des transitions valides :

- TC1:
E1 –(T1)→ **E2** –(T1) → **E2** –(T2[somme>=prix]) → **E4** –(T3) → **E1**
- TC2 :
E1 –(T1) → **E2** –(T2[somme<prix]) → **E3** –(T1[somme>=prix]) → **E4** –(T3) → **E1**
- TC3 :
E1 –(T2) → **E3** –(T1[somme<prix]) → **E3** –(T1[somme>=prix]) → **E4** –(T3) → **E1**

3. Transitions invalides

Mais il est possible (et quelquefois souhaitable, d'aller plus loin et de s'intéresser aux transitions invalides.

Nous pouvons donc enrichir la couverture avec les transitions invalides.

Mais c'est quoi une transition invalide?

C'est une transition qui ne devrait pas arriver sur un état.

Sur un graphe, elles ne sont pas représentées.

Dans la table de transition d'état, elles correspondent aux cases vides (ou marquées d'un **X**).

Ces transitions invalides doivent particulièrement nous intéresser car elles peuvent révéler des besoins de conception de logiciel (voir système) particuliers.

Par exemple:

- notre distributeur doit assurer de ne jamais envoyer l'événement « distribution produit » quand il se trouve dans les états E1, E2 et E3.
- notre distributeur doit empêcher l'utilisateur de mettre des pièces pendant la préparation du produit.

4. Couverture 1-switch

Il peut être également utile (hors programme dans ISTQB) d'atteindre une couverture de 100% d'aiguillage-1, ou couverture de 1-switch.

La **couverture de 100% d'aiguillage-1** (ou **couverture de 1-switch**) signifie que **chaque séquence valide de deux transitions successives est testée au moins une fois**.

Autrement dit, on teste chaque enchaînement possible entre deux transitions pour s'assurer que toutes les paires de transitions consécutives sont couvertes.

Cette couverture est plus complète que la couverture des transitions simples (ou 0-switch), qui ne garantit que le test de chaque transition individuelle.

En pratique, la couverture de 1-switch permet de détecter certains types de défauts qui ne seraient pas détectés uniquement avec la couverture de transitions simples, en mettant en lumière des problèmes potentiels dans la manière dont les états se suivent dans le système.

Il faudra dans ce cas poser dans un tableau la liste des séquences de 2 transitions valides successives. Dans notre exemple, cela donne:

1. E1 → E2 → E2
2. E1 → E2 → E4
3. E1 → E2 → E3
4. E1 → E3 → E3
5. E1 → E3 → E4
6. E2 → E4 → E1
7. E2 → E3 → E4
8. E2 → E3 → E3
9. E3 → E4 → E1
10. E3 → E3 → E4
11. E4 → E1 → E2
12. E4 → E1 → E3

Voici une proposition de cas de test qui permettrait d'atteindre cette couverture:

- test 1: E1 → E2 → E2 → E4 → E1 → E3 → E4 → E1 qui couvre les séquences 1, 6, 7, 9 et 12.
- test 2 : E1 → E2 → E4 → E1 → E2 → E3 → E3 → E4 → E1 qui couvre les séquences 2, 3, 8, 9 et 11

- test 3 : E1 → E3 → E3→F4 → E1 → E3 → F4 → E1 qui couvre les séquences 4, 5, 9, 10 et 12

À partir d'un graphe ou de la table, nous pouvons aussi voir des omissions dans les états et transitions pouvant rendre notre distributeur peu robuste:

- l'utilisateur ne peut pas annuler sa demande: Gênant dans le cas où il n'a pas assez de monnaie pour payer ! Cela soulèvera un problème à minima moral, voire juridique de conserver son argent sans lui rendre le service payé.
- l'utilisateur peut abandonner sa demande après avoir sélectionné le prix et s'en aller. Le distributeur reste bloqué en attente de paiement jusqu'à ce qu'un autre utilisateur ait envie du même produit que l'utilisateur précédent.

Il va de soi qu'il faudra surement améliorer notre distributeur car il y a un risque qu'aucun client ne l'utilise...